POC Title: Local-First Collaborative Notes using CRDT (Java)

Objective:
Build a Java-based Proof of Concept that allows multiple clients to edit a shared text offline and later synchronize changes without conflicts, even when operations arrive out of order (including delete-before-insert scenarios).

Why this POC:
Traditional systems fail when concurrent edits happen offline. This POC demonstrates conflict-free synchronization using CRDT principles, making it suitable for distributed systems, offline-first apps, and real-time collaboration tools.

Core Concepts:
• Operation-based synchronization (Op-log)
• Lamport timestamps for deterministic ordering
• Anchor-based inserts instead of index-based inserts
• Tombstones instead of hard deletes
• Pending tombstones to handle delete-before-insert

Architecture Overview:
Client → Server (HTTP)
Server maintains operation log and document state in memory.

Key Data Models:
OpId: lamport:clientId:sequence
Operation: insert / delete with metadata
Element: Represents a character/node in document with tombstone flag

Algorithm Flow:
1. Client sends insert/delete operation
2. Server assigns serverVersion
3. Insert operations create elements linked to anchors
4. Delete operations mark tombstones or are stored as pending if target not found
5. When insert arrives after delete, pending tombstone is applied immediately
6. Document is rendered via DFS traversal starting from HEAD

API Endpoints:
POST /op – Submit an operation
GET /sync?since=N – Fetch operations after version N
GET /doc – Rendered document text
GET /health – Server health and state
GET /demo – Runs delete-before-insert demo

Acceptance Criteria:
• Insert ordering is deterministic
• Delete-before-insert is handled correctly
• Delta sync works correctly
• Final document is consistent across syncs

Demo Scenario:
1. Delete operation arrives first
2. Insert operation arrives later
3. Insert-after-deleted-anchor occurs
Final Output must be: "i"

Developer Tasks:
• Build Java HTTP server

• Implement OpId ordering
• Implement CRDT document model
• Handle pending tombstones
• Implement delta sync
• Provide demo endpoint

Out of Scope:
• Authentication
• Database persistence
• WebSocket / real-time UI
• Production hardening

Expected Outcome:
A runnable Java POC proving feasibility of offline-first collaborative editing with conflict-free synchronization.

Usage:
Compile and run the provided Java file. Use curl or browser to test endpoints.

This document can be directly assigned to a developer as a POC task.